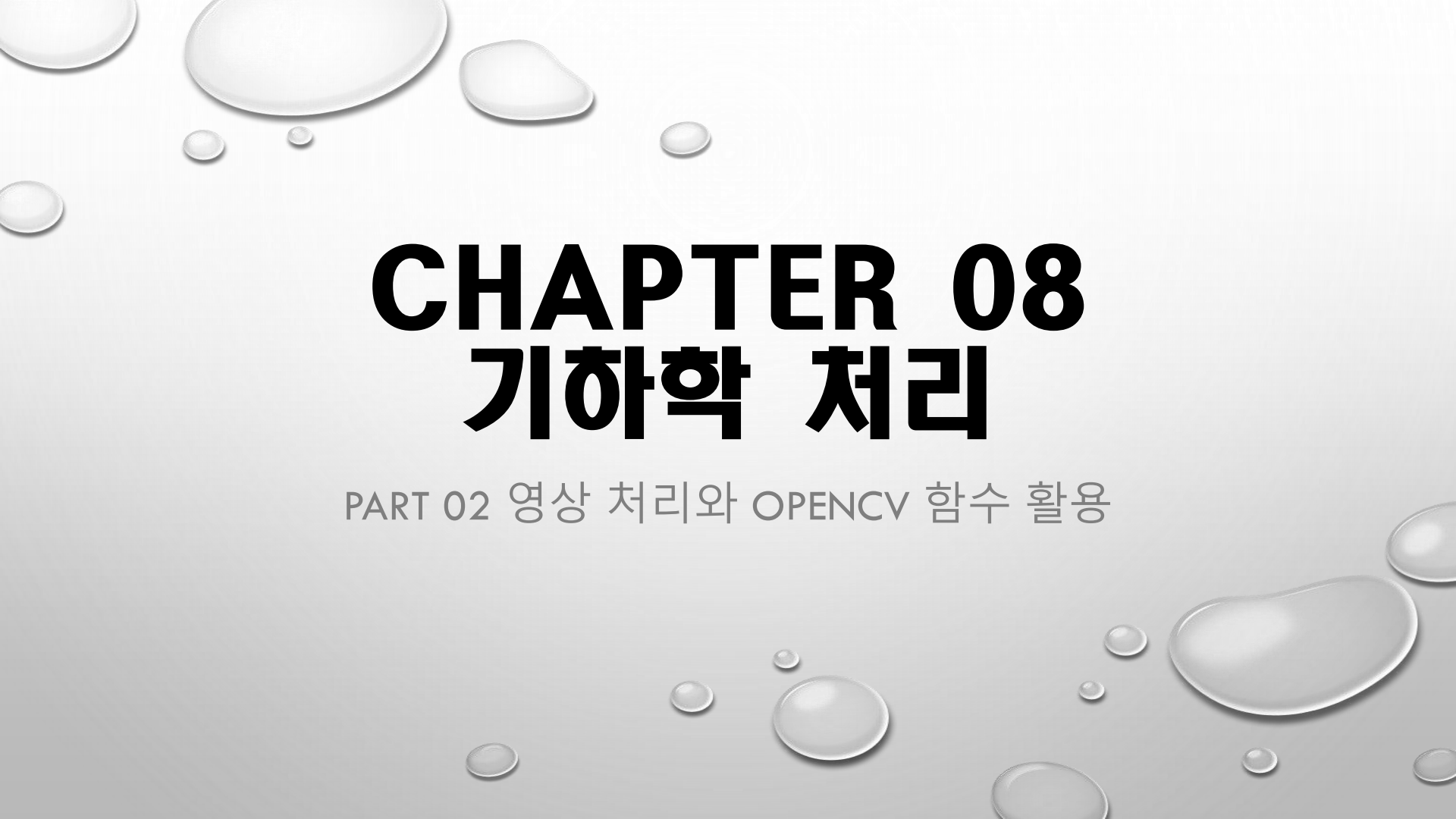


The background of the slide is a light gray gradient, decorated with numerous realistic water droplets of various sizes. Some droplets are in the top left corner, others are scattered along the bottom edge, and a few are on the right side. The droplets have highlights and shadows, giving them a three-dimensional appearance.

CHAPTER 08

기하학 처리

PART 02 영상 처리와 OPENCV 함수 활용

CONTENTS

8.1 사상

8.2 크기 변경(확대/축소)

8.3 보간

8.4 평행이동

8.5 회전

8.6 행렬 연산을 통한 기하학 변환 - 어파인 변환

8.7 원근 투시(투영) 변환

8.1 사상

◆ 기하학적 처리의 기본

❖ 화소들의 배치 변경 → 사상의 의미 이해

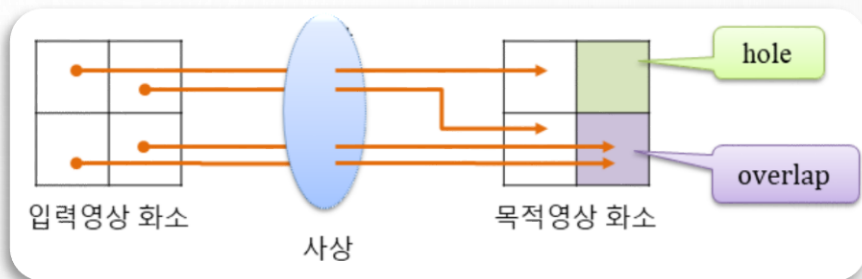
◆ 사상

- 가상 주소와 물리 주소의 대응 관계, 가상 주소로부터 물리 주소를 찾아내는 일
- 영상처리
 - ✓ 화소들의 배치를 변경할 때, 입력 영상의 좌표와 목적 영상의 좌표의 대응 관계를 찾는 것
 - ✓ 순방향 사상 / 역방향 사상

8.1 사상

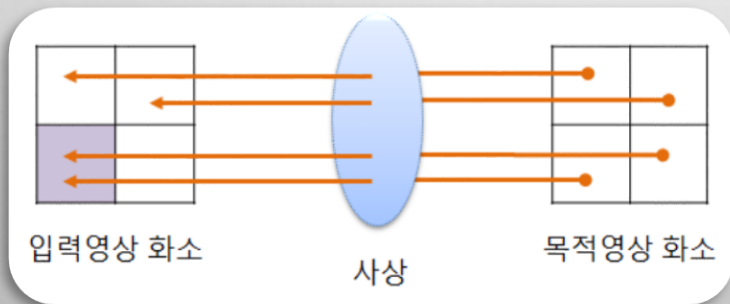
◆ 순방향 사상 (x, y 로 x', y' 를 계산)

- ❖ 원본 영상의 좌표를 중심으로 목적 영상의 좌표를 계산하여 화소의 위치를 변환하는 방식
- ❖ 홀(hole)이나 오버랩(overlap)의 문제가 발생



◆ 역방향 사상 (x', y' 로 x, y 를 계산)

- ❖ 목적 영상의 좌표를 인덱싱(순회)하여 입력 영상의 좌표를 찾아는 방식
- ❖ 홀이나 오버랩은 발생하지 않음
- ❖ 입력 영상의 한 화소를 목적 영상의 여러 화소에서 사용될 수 있음



8.2 크기 변경 (확대/축소)

◆ 크기 변경(*scaling*)

- ❖ 입력 영상의 가로와 세로로 크기를 변경해서 목적 영상을 만드는 방법
 - 확대 – 입력 영상보다 목적 영상의 크기가 커짐
 - 축소 – 입력 영상보다 목적 영상의 크기가 작아짐

◆ 크기 변경 수식

- ❖ 변경 비율 및 변경 크기 이용

$$\begin{aligned}x' &= x \cdot \text{ratio } X \\ y' &= y \cdot \text{ratio } Y\end{aligned}$$

$$\text{ratio } X = \frac{dst_{width}}{org_{width}}, \quad \text{ratio } Y = \frac{dst_{height}}{org_{height}}$$

8.2 크기 변경 (확대/축소)

예제 8.2.1

영상 크기 변경 - 01.scaling.cpp

```
01 import numpy as np, cv2
02
03 def scaling(img, size):                                # 크기 변경 함수
04     dst = np.zeros(size[::-1], img.dtype)            # size와 shape는 원소 역순
05     ratioY, ratioX = np.divide(size[::-1], img.shape[:2]) # 비율 계산
06     y = np.arange(0, img.shape[0], 1)                # 입력 영상 세로(y) 좌표 생성
07     x = np.arange(0, img.shape[1], 1)                # 입력 영상 가로(x) 좌표 생성
08     y, x = np.meshgrid(y, x)                          # i, j 좌표에 대한 정방행렬 생성
09     i, j = np.int32(y * ratioY), np.int32(x * ratioX) # 목적 영상 좌표
10     dst[i, j] = img[y, x]                             # 정방향 사상→목적 영상 좌표 계산
11     return dst
```

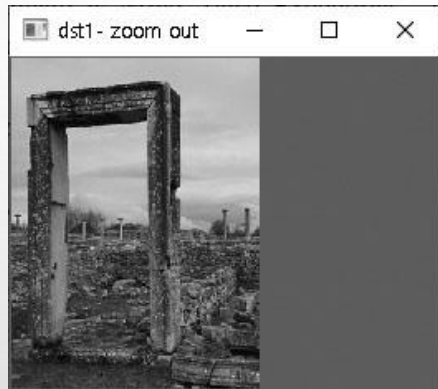
```
13 def scaling2(img, size):                              # 크기 변경 함수
14     dst = np.zeros(size[::-1], img.dtype)
15     ratioY, ratioX = np.divide(size[::-1], img.shape[:2])
16     for y in range(img.shape[0]):                      # 입력 영상 순회
17         for x in range(img.shape[1]):
18             i, j = int(y * ratioY), int(x * ratioX) # 목적 영상의 y, x 좌표
19             dst[i, j] = img[y, x]
20     return dst
```

8.2 크기 변경 (확대/축소)

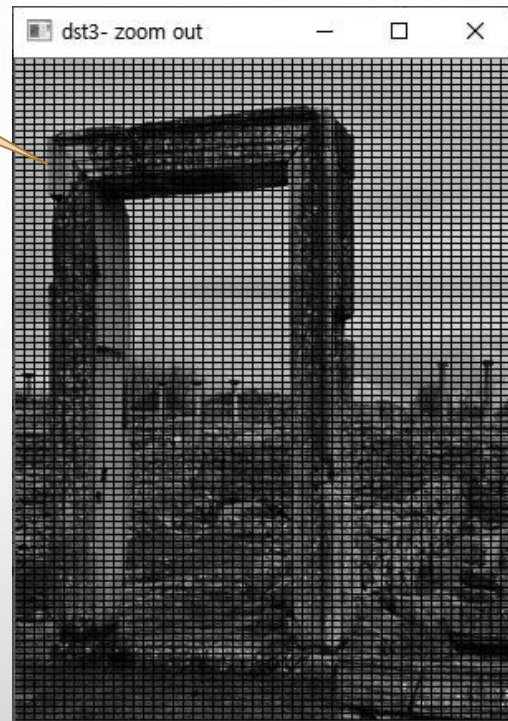
```
22 def time_check(func, image, size, title):                # 수행시간 체크 함수
23     start_time = time.perf_counter()
24     ret_img = func(image, size)
25     elapsed = (time.perf_counter() - start_time) * 1000
26     print(title, "수행시간 = %0.2f ms" % elapsed)
27     return ret_img
28
29 image = cv2.imread("images/scaling.jpg", cv2.IMREAD_GRAYSCALE)
30 if image is None: raise Exception("영상파일 읽기 에러")
31
32 dst1 = scaling(image, (150, 200))                        # 크기 변경- 축소
33 dst2 = scaling2(image, (150, 200))                       # 크기 변경- 축소
34 dst3 = time_check(scaling, image, (300,400), "[방법1]: 정방행렬 방식>")
35 dst4 = time_check(scaling2, image, (300,400), "[방법2]: 반복문 방식>")
36
37 cv2.imshow("image", image)
38 cv2.imshow("dst1- zoom out", dst1)
39 cv2.imshow("dst3- zoom out" , dst3)
40 cv2.resizeWindow("dst1- zoom out", 260, 200)            # 윈도우 크기 확장
41 cv2.waitKey(0)
```


8.2 크기 변경 (확대/축소)

◆ 실행결과



홀의 발생으로 결과영상 품질 저하



Run: 01.scaling

C:\Python\python.exe D:/source/chap08/01.scaling.py

[방법1] 정방향 필터 방식> 수행시간 = 2.67 ms

[방법2] 반복문 방식> 수행시간 = 188.34 ms

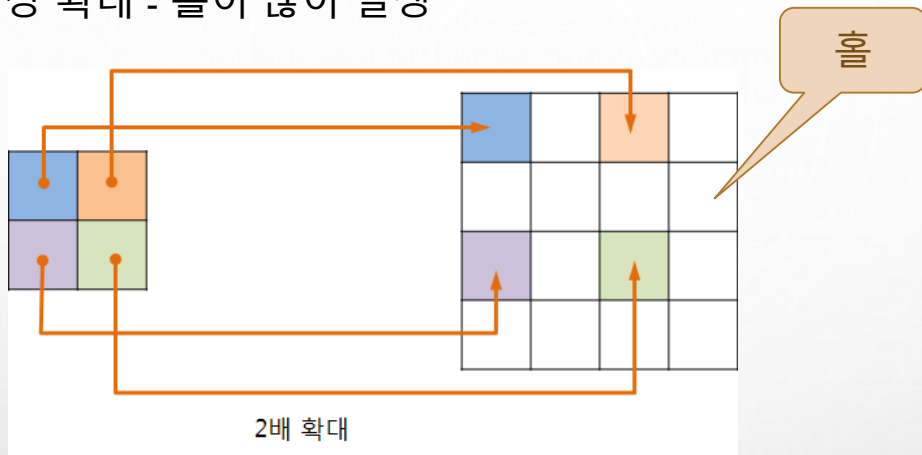
8.3 보간

◆ 8.3.1 최근접 이웃 보간법

8.3.2 양선형 보간법

8.3 보간

◆ 순방향 사상으로 영상 확대 - 홀이 많이 발생



◆ 역방향 사상을 통해서 홀의 화소들을 입력영상에서 찾음

❖ 영상을 축소할 때에는 오버랩의 문제가 발생

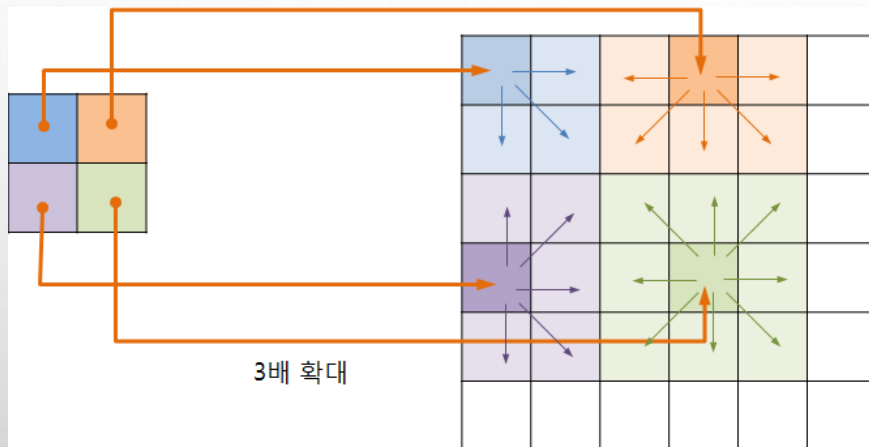
◆ 보간법 필요

❖ 목적영상에서 홀의 화소들을 채우고

❖ 오버랩이 되지 않게 화소들을 배치하여 목적영상을 만드는 기법

8.3.1 최근접 이웃 보간법

- ◆ 홀이 되어 할당 받지 못하는 화소들의 값을 찾을 때,
- ◆ 목적 영상의 화소에 가장 가깝게 이웃한 입력 영상의 화소값을 찾는 방법



$$\begin{aligned} y' &= y \cdot \text{ratio } Y \\ x' &= x \cdot \text{ratio } X \end{aligned} \Rightarrow y = \frac{y'}{\text{ratio } Y}, x = \frac{x'}{\text{ratio } X}$$

8.3.1 최근접 이웃 보간법

예제 8.3.1 크기변경 & 최근접 이웃 보간- 02.scaling_nearest.cpp

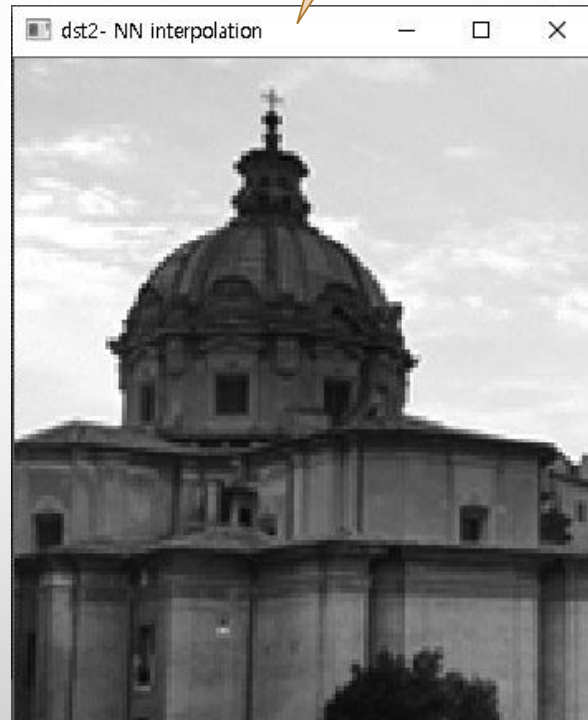
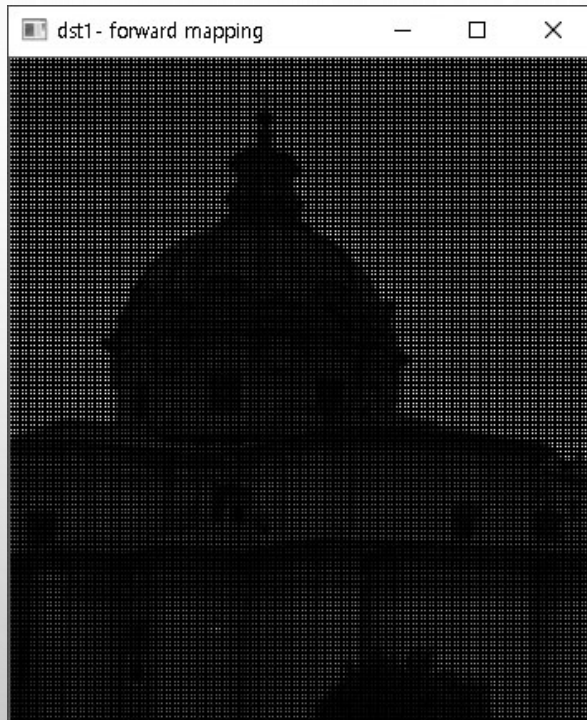
```
01 import numpy as np, cv2
02 from Common.interpolation import scaling    # interpolation 모듈의 scaling() 함수 импорт
03
04 def scaling_nearest(img, size):              # 크기 변경 함수3
05     dst = np.zeros(size[::-1], img.dtype)   # 행렬과 크기는 원소가 역순
06     ratioY, ratioX = np.divide(size[::-1], img.shape[:2]) # 변경 크기 비율
07     i = np.arange(0, size[1], 1)            # 목적 영상 세로(i) 좌표 생성
08     j = np.arange(0, size[0], 1)            # 목적 영상 가로(j) 좌표 생성
09     i, j = np.meshgrid(i, j)
10     y, x = np.int32(i / ratioY), np.int32(j / ratioX) # 입력 영상 좌표
11     dst[i, j] = img[y, x]                   # 역방향 사상 → 입력 영상 좌표
12
13     return dst

15 image = cv2.imread("images/interpolation.jpg", cv2.IMREAD_GRAYSCALE)
16 if image is None: raise Exception("영상파일 읽기 에러")
17
18 dst1 = scaling(image, (350, 400))           # 크기 변경- 기본
19 dst2 = scaling_nearest(image, (350, 400))   # 크기 변경- 최근접 이웃 보간
20
21 cv2.imshow("image", image)
22 cv2.imshow("dst1- forward mapping", dst1)    # 순방향 사상
23 cv2.imshow("dst2- NN interpolation", dst2)   # 최근접 이웃 보간
24 cv2.waitKey(0)
```

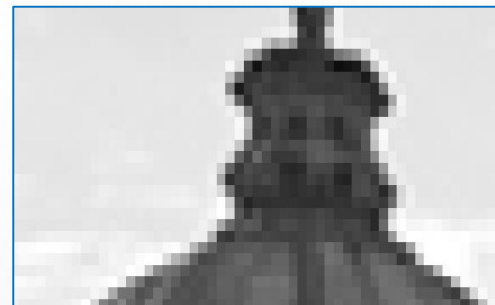
8.3.1 최근접 이웃 보간법

최근접 이웃 보간법

◆ 실행결과



8.3.2 양선형 보간법

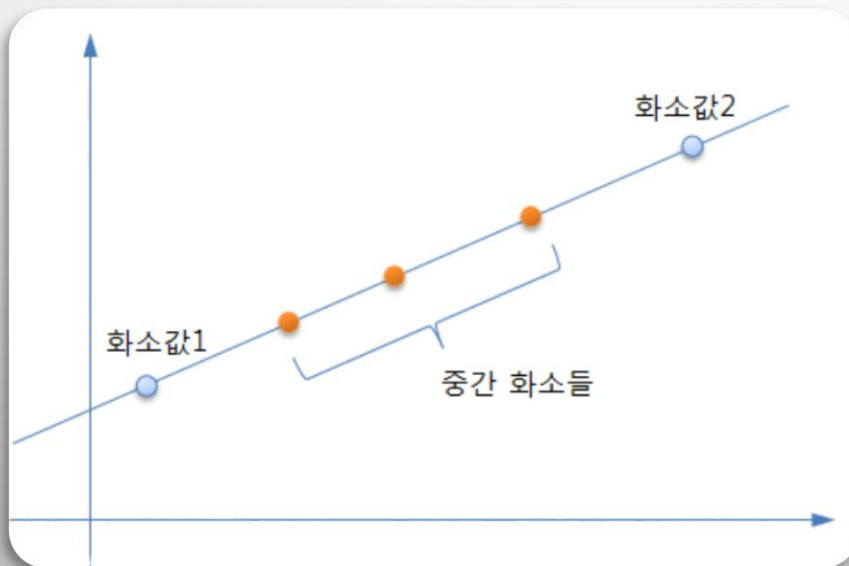


◆ 최근접 이웃 보간법

- ❖ 확대비율이 커지면, 모자이크 현상 혹은 경계부분에서 계단현상 발생

◆ 양선형 보간으로 해결

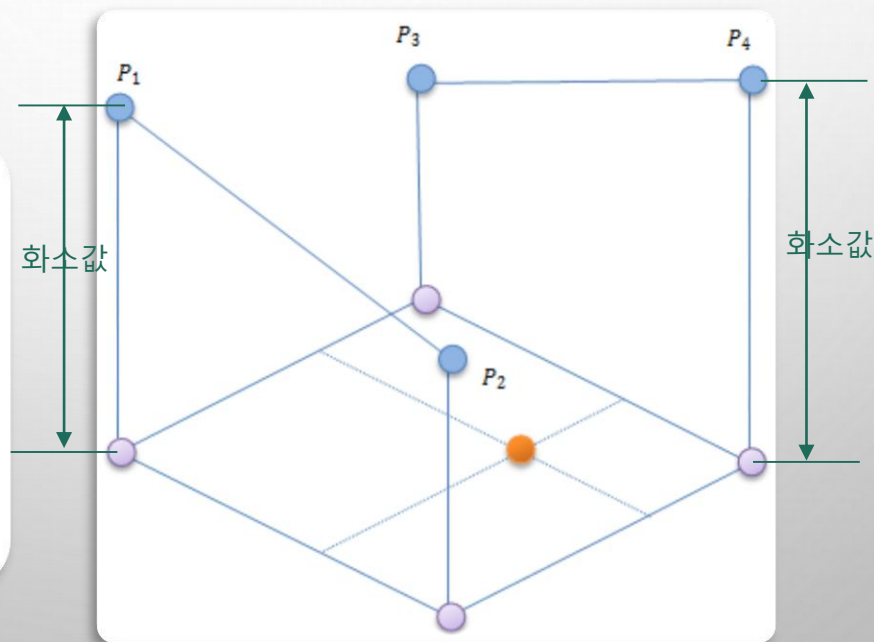
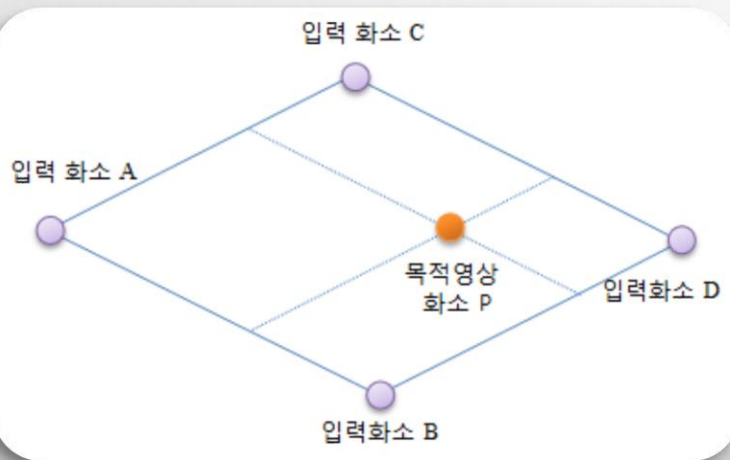
- ❖ 직선의 선상에 위치한 중간 화소들의 값은 직선의 수식을 이용해서 쉽게 계산



8.3.2 양선형 보간법

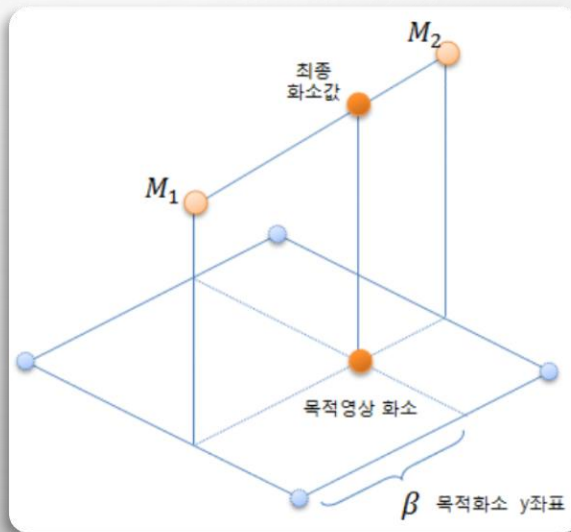
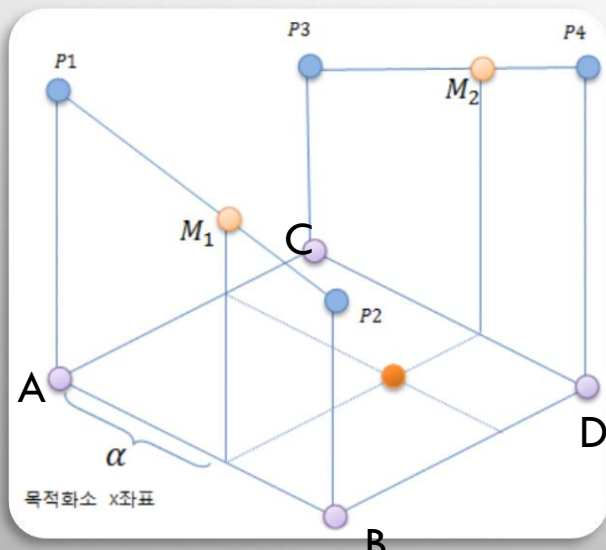
◆ 양선형 보간법 - 선형 보간을 두 번에 걸쳐서 수행하기에 붙여진 이름

- ❖ 목적영상 화소(P)를 계산(역변환)하여 입력영상의 4개 화소(A, B, C, D) 값 가져옴
- ❖ 4개 화소를 두 개씩(AB, CD) 묶어서 두 화소를 화소값(P_1, P_2, P_3, P_4)으로 잇는 직선 구성



8.3.2 양선형 보간법

- ❖ 직선상에서 목적영상 화소의 좌표로 중간 위치 찾을
 - 중간 위치는 거리 비율(α , $1-\alpha$)로 찾을
- ❖ 중간 위치 화소값(M_1 , M_2) 계산
 - 입력 화소값과 거리 비율(α , $1-\alpha$)로 직선 수식 이용해 계산
- ❖ 두 개의 중간 화소값(M_1 , M_2)을 잇는 직선 다시 구성
- ❖ 두 개의 중간 화소값과 거리 비율(β , $1-\beta$)로 직선 수식을 이용해서 최종 화소값 계산



8.3.2 양선형 보간법

◆ 수식 정리

$$M_1 = \alpha \cdot B + (1 - \alpha) \cdot A = A + \alpha \cdot (B - A)$$

$$M_2 = \alpha \cdot D + (1 - \alpha) \cdot C = C + \alpha \cdot (D - C)$$

$$P = \beta \cdot M_2 + (1 - \beta) \cdot M_1 = M_1 + \beta \cdot (M_2 - M_1)$$

◆ OpenCV 보간 옵션

- `cv::resize()`, `cv::remap()`, `cv::warpAffine()`, `cv::warpPerspective()` 등의 함수에서 사용

〈표 8.3.1〉 보간 방법에 대한 flag 옵션

옵션 상수	값	설명
INTER_NEAREST	0	최근접 이웃 보간
INTER_LINEAR	1	양선형 보간 (기본값)
INTER_CUBIC	2	바이큐빅 보간 - 4x4 이웃 화소 이용
INTER_AREA	3	픽셀 영역의 관계로 리샘플링
INTER_LANCZOS4	4	Lanczos 보간 - 8x8 이웃 화소 이용

예제 8.3.2 크기변경 & 양선형 보간 - 03.scaling_bilinear.cpp

```
01 import numpy as np, cv2
02 from Common.interpolation import scaling_nearest # 최근접 이웃 보간 함수 임포트
03
04 def bilinear_value(img, pt): # 단일 화소 양선형 보간 수행
05     x, y = np.int32(pt)
06     if x >= img.shape[1]-1: x = x - 1 # 영상 범위 벗어남 처리
07     if y >= img.shape[0]-1: y = y - 1
08
09     P1, P2, P3, P4 = np.float32(img[y:y+2,x:x+2].flatten()) # 4개 화소-관심 영역으로
10     ## 4개 화소 - 화소 직접 접근
11     # P1 = float(img[y, x]) # 좌상단 화소
12     # P2 = float(img[y + 0, x + 1]) # 우상단 화소
13     # P3 = float(img[y + 1, x + 0]) # 좌하단 화소
14     # P4 = float(img[y + 1, x + 1]) # 우하단 화소
15
16     alpha, beta = pt[1] - y, pt[0] - x # 거리 비율
17     M1 = P1 + alpha * (P3 - P1) # 1차 보간
18     M2 = P2 + alpha * (P4 - P2)
19     P = M1 + beta * (M2 - M1) # 2차 보간
20     return np.clip(P, 0, 255) # 화소값 saturation 후 반환
```

8.3.2 양선형 보간법

```
22 def scaling_bilinear(img, size):                                # 양선형 보간
23     ratioY, ratioX = np.divide(size[::-1], img.shape[:2]) # 변경 크기 비율-행태와 크
24
25     dst = [[ bilinear_value(img, (j/ratioX, i/ratioY)) # 리스트 생성
26               for j in range(size[0])]
27             for i in range(size[1])]
28     return np.array(dst, img.dtype)
29
30 image = cv2.imread("images/interpolation.jpg", cv2.IMREAD_GRAYSCALE)
31 if image is None: raise Exception("영상파일 읽기 에러")
32
33 size = (350, 400)
34 dst1 = scaling_bilinear(image, size)                            # 크기 변경- 양선형 보간
35 dst2 = scaling_nearest(image, size)                             # 크기 변경- 최근접 이웃
36 dst3 = cv2.resize(image, size, 0, 0, cv2.INTER_LINEAR) # OpenCV 함수 - 양선형
37 dst4 = cv2.resize(image, size, 0, 0, cv2.INTER_NEAREST) # OpenCV 함수 - 최근접
38
39 cv2.imshow("image", image)
40 cv2.imshow("User_bilinear", dst1);
41 cv2.imshow("User_Nearest", dst2)
```

8.3.2 양선형 보간법

◆ 실행결과



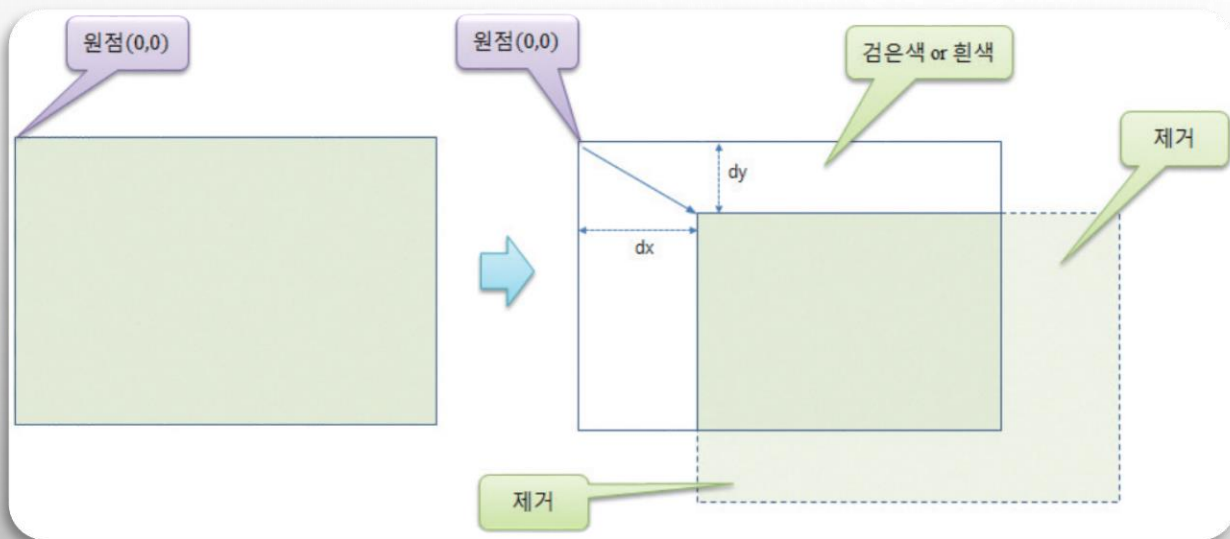
입력영상



계산 현상의 감소

8.4 평행이동

- ❖ 영상의 원점을 기준으로 모든 화소를 동일하게 가로방향과 세로 방향으로 옮기는 것
- ❖ 가로 방향으로 dx 만큼, 세로 방향으로 dy 만큼 전체 영상의 모든 화소 이동한 예



순방향 사상

$$\begin{aligned}x' &= x + dx \\ y' &= y + dy\end{aligned}$$

역방향 사상

$$\begin{aligned}x &= x' - dx \\ y &= y' - dy\end{aligned}$$

8.4

예제 8.4.1 영상 평행이동 - 04.translation.cpp

```
01 import numpy as np, cv2
02
03 def contain(p, shape):
04     return 0<= p[0] < shape[0] and 0<= p[1] < shape[1]
05
06 def translate(img, pt):
07     dst = np.zeros(img.shape, img.dtype)
08     for i in range(img.shape[0]):
09         for j in range(img.shape[1]):
10             x, y = np.subtract((j, i), pt)
11             if contain((y, x), img.shape):
12                 dst[i, j] = img[y, x]
13     return dst
```

```
14
15 image = cv2.imread("images/translate.jpg", cv2.IMREAD_GRAYSCALE)
16 if image is None: raise Exception("영상파일 읽기 에러")
17
18 dst1 = translate(image, (30, 80)) # x=30, y=80
19 dst2 = translate(image, (-70, -50))
20
21 cv2.imshow("image", image)
22 cv2.imshow("dst1: transted to (30, 80)", dst1);
23 cv2.imshow("dst2: transted to (-70, -50)", dst2);
24 cv2.waitKey(0)
```

목적 영상 생성

목적 영상 순화-역방향 사상

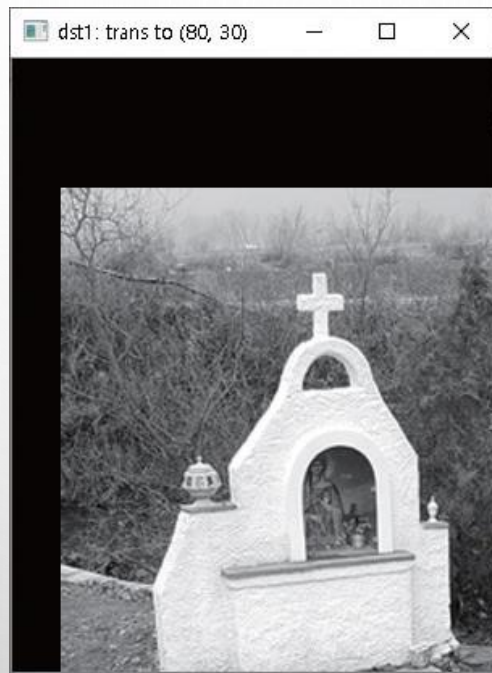
좌표는 가로, 세로 순서

영상 범위 확인

행렬은 행, 열 순서

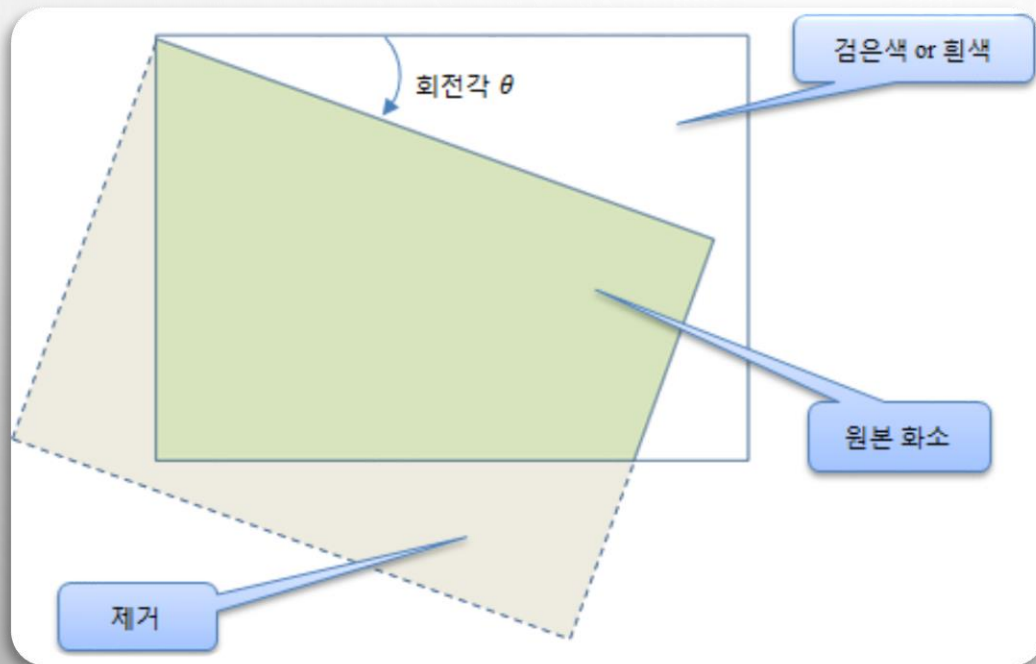
8.4 평행이동

◆ 실행결과



8.5 회전

- ◆ 원점을 기준으로 입력 영상의 모든 화소를 영상의 원하는 각도만큼 모든 화소에 대해서 회전 변환을 시키는 것
 - ❖ 직교 좌표계에서 회전 변환은 반시계 방향으로 적용
 - ❖ 영상 좌표계에서는 y 좌표가 하단으로 내려갈수록 증가하기 때문에 시계방향 회전



8.5 회전

◆ 회전 변환 수식

순방향 사상

$$x' = x \cdot \cos \theta - y \cdot \sin \theta$$

$$y' = x \cdot \sin \theta + y \cdot \cos \theta$$

역방향사상

$$x = x' \cdot \cos \theta + y' \cdot \sin \theta$$

$$y = -x' \cdot \sin \theta + y' \cdot \cos \theta$$

◆ 특정 좌표에서(center X, center Y) 회전하는 경우

- ❖ 회전의 기준점으로 영상을 이동시킨 후, 회전 수행
- ❖ 다시 원점 좌표로 이동

$$x = (x' - \text{center } X) \cos \theta + (y' - \text{center } Y) \sin \theta + \text{center } X$$

$$y = -(x' - \text{center } X) \sin \theta + (y' - \text{center } Y) \cos \theta + \text{center } Y$$

8.5 회전

예제 8.5.1 영상 회전 - 05.rotation.py

```
01 import numpy as np, math, cv2
02 from Common.interpolation import bilinear_value      # 양선형 보간 함수 импорт
03 from Common.functions import contain                # 사각형으로 범위 확인 함수
04
05 def rotate(img, degree):                            # 원점 기준 회전 변환 함수
06     dst = np.zeros(img.shape[:2], img.dtype)        # 목적 영상 생성
07     radian = (degree/180) * np.pi                  # 회전 각도- 라디언
08     sin, cos = np.sin(radian), np.cos(radian)       # 사인, 코사인 값 미리 계산
09
10     for i in range(img.shape[0]):                   # 목적 영상 순회- 역방향 사상
11         for j in range(img.shape[1]):
12             y = -j * sin + i * cos
13             x = j * cos + i * sin                    # 회전 변환 수식
14             if contain((y, x), img.shape):          # 입력 영상의 범위 확인
15                 dst[i, j] = bilinear_value(img, [x, y]) # 화소값 양선형 보간
16     return dst
```

8.5 회전

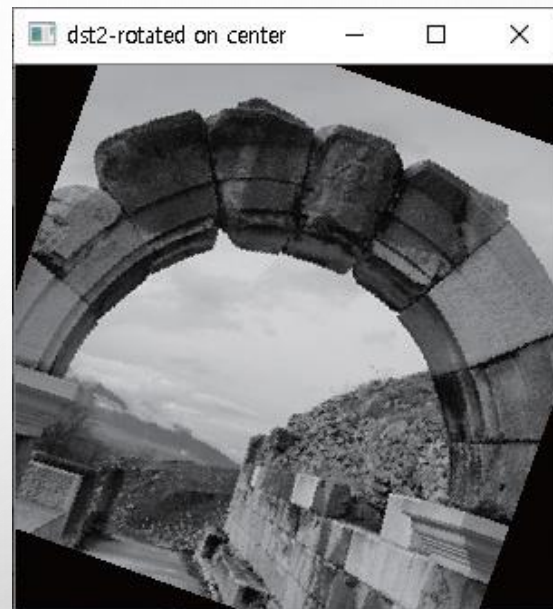
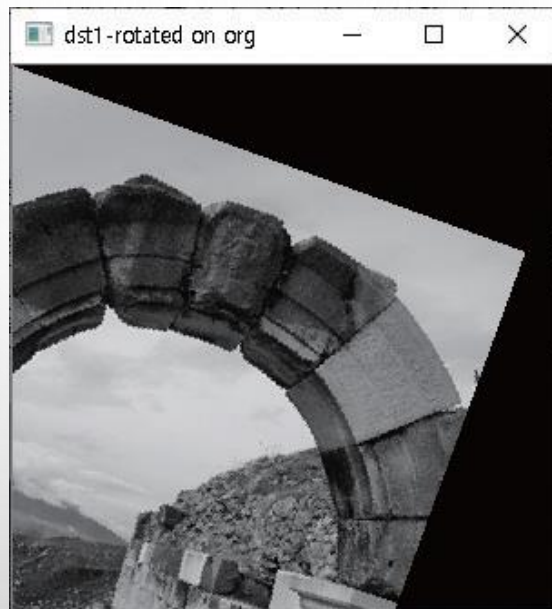
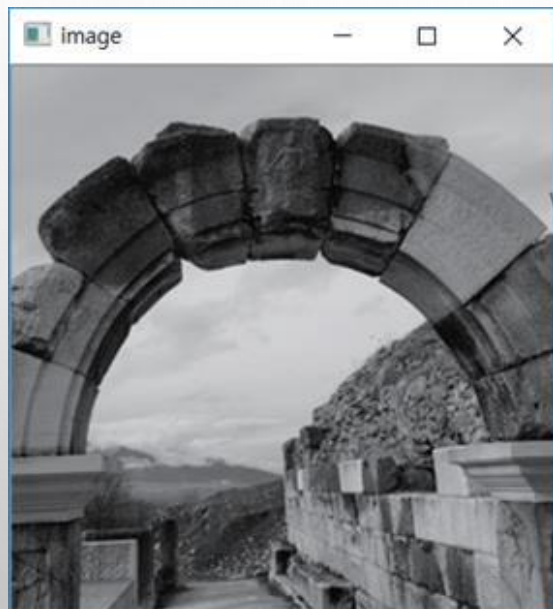
```
18 def rotate_pt(img, degree, pt):           # pt 기준 회전 변환 함수
19     dst = np.zeros(img.shape[:2], img.dtype) # 목적 영상 생성
20     radian = (degree/180) * np.pi          # 회전 각도- 라디언
21     sin, cos = math.sin(radian), math.cos(radian) # 사인, 코사인 값 미리 계산
22
23     for i in range(img.shape[0]):           # 목적 영상 순화- 역방향 사상
24         for j in range(img.shape[1]):
25             jj, ii = np.subtract((j, i), pt) # 중심 좌표로 평행이동
26             y = -jj * sin + ii * cos          # 회전 변환 수식
27             x = jj * cos + ii * sin
28             x, y = np.add((x, y), pt)         # 중심 좌표로 평행이동
29             if contain((y, x), img.shape):    # 입력 영상의 범위 확인
30                 dst[i, j] = bilinear_value(img, (x, y)) # 화소값 양선형 보간
31     return dst
```

8.5 회전

```
33 image = cv2.imread("images/rotate.jpg", cv2.IMREAD_GRAYSCALE)
34 if image is None: raise Exception("영상파일 읽기 에러")
35
36 center = np.divmod(image.shape[:: -1], 2)[0]           # 영상 크기로 중심 좌표 계산
37 dst1 = rotate(image, 20)                                # 원점 기준 회전 변환
38 dst2 = rotate_pt(image, 20, center)                     # center 기준 회전 변환
39
40 cv2.imshow("image", image)
41 cv2.imshow("dst1 : rotated on (0, 0)", dst1);
42 cv2.imshow("dst2 : rotated on center point", dst2);
43 cv2.waitKey(0)
```

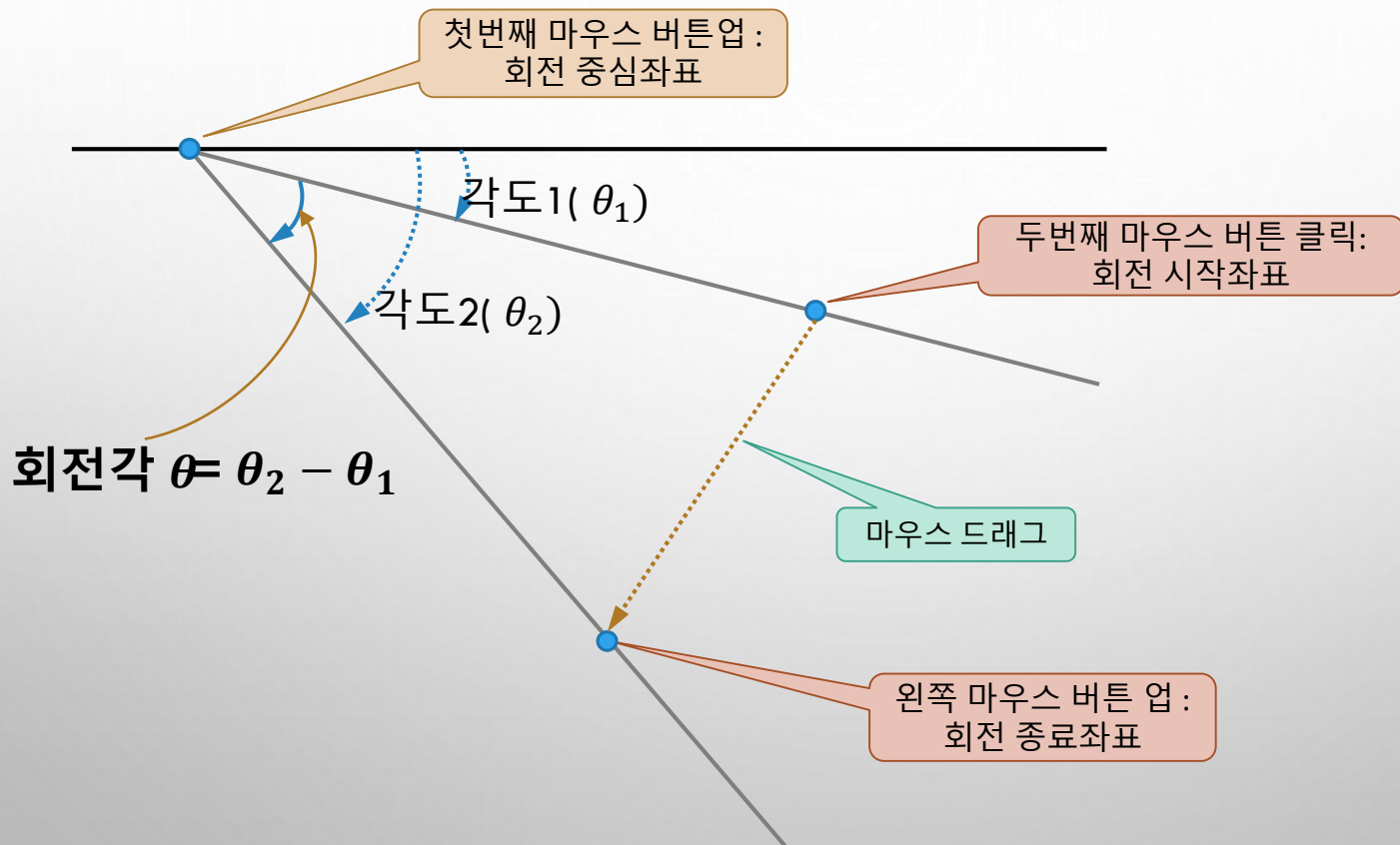

8.5 회전

◆ 실행결과



8.5 회전 - 심화예제

◆ 심화예제 - 기준점에서 마우스 드래그로 회전 수행하기



8.5 회전 - 심화예제

◆ 심화예제 - 기준점에서 마우스 드래그로 회전 수행하기

심화예제 8.5.2

마우스 드래그로 영상 회전하기 - 06.rotation2.py

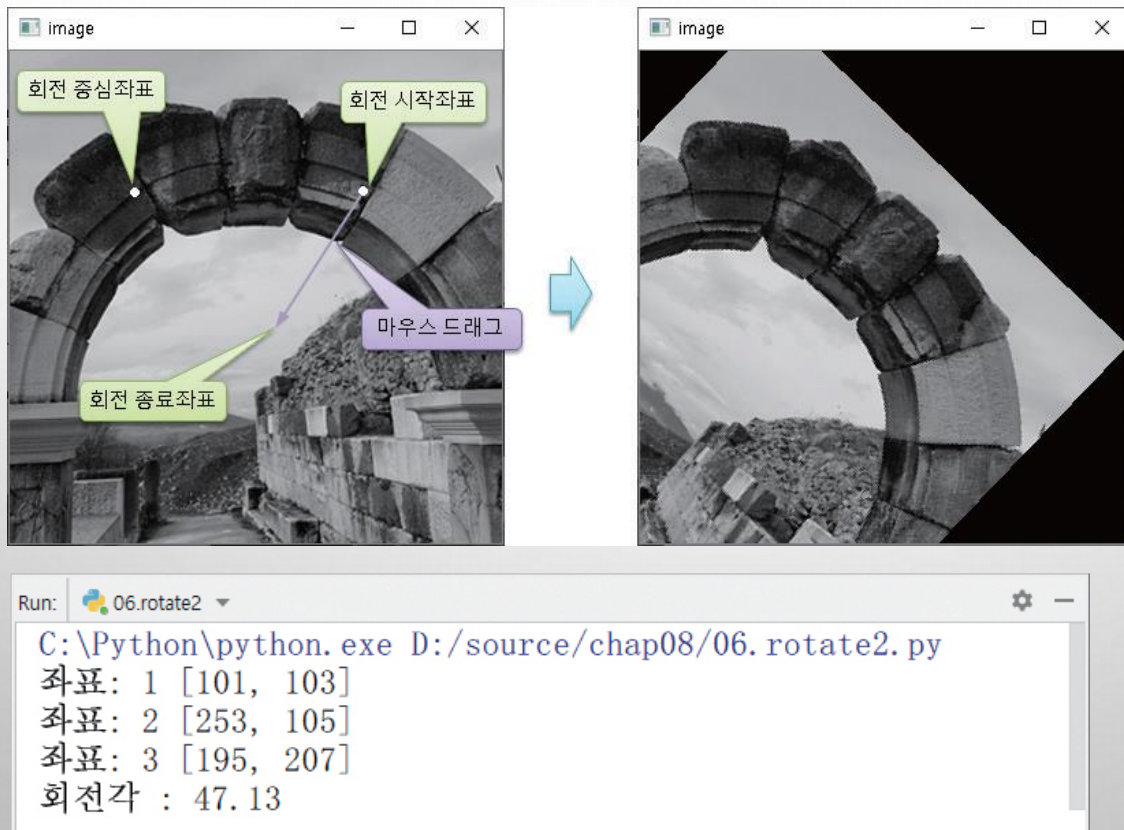
```
01 import numpy as np, cv2
02 from Common.interpolation import rotate_pt          # 좌표 기준 영상 회전 함수 임포트
03
04 def calc_angle(pt):                                  # 3개 좌표로 각도 계산 함수
05     d1 = np.subtract(pts[1], pts[0]).astype(float)  # 두 좌표간 차분 계산
06     d2 = np.subtract(pts[2], pts[0]).astype(float)
07     angle1 = cv2.fastAtan2(d1[1], d1[0])             # 차분으로 각도 계산
08     angle2 = cv2.fastAtan2(d2[1], d2[0])
09     return (angle2 - angle1)                          # 두 각도 간의 차분
10
11 def draw_point(x, y):                                # 좌표 저장 및 그리기
12     pts.append([x,y])
13     print("좌표:", len(pts), [x,y])                  # 클릭 좌표 표시
14     cv2.circle(tmp, (x, y), 2, 255, 2)              # 중심 좌표 표시
15     cv2.imshow("image", tmp)
```

8.5 회전 - 심화예제

```
17 def onMouse(event, x, y, flags, param):           # 마우스 콜백 함수
18     global tmp, pts
19     if (event == cv2.EVENT_LBUTTONUP and len(pts) == 0): draw_point(x, y)
20     if (event == cv2.EVENT_LBUTTONDOWN and len(pts) == 1): draw_point(x, y)
21     if (event == cv2.EVENT_LBUTTONUP and len(pts) == 2): draw_point(x, y)
22
23     if len(pts) == 3:
24         angle = calc_angle(pts)                   # 회전각 계산
25         print("회전각: %3.2f" % angle)
26         dst = rotate_pt(image, angle, pts[0])      # 저자 구현 회전 수행
27         cv2.imshow("image", dst)
28         tmp = np.copy(image)                       # 임시 행렬 초기화
29         pts = []                                   # 클릭 좌표 초기화
30
31 image = cv2.imread("images/rotate.jpg", cv2.IMREAD_GRAYSCALE)
32 if image is None: raise Exception("영상파일 읽기 에러")
33 tmp = np.copy(image)
34 pts = []
35
36 cv2.imshow("image", image)
37 cv2.setMouseCallback("image", onMouse, 0)
38 cv2.waitKey(0)
```

8.5 회전 - 심화예제

◆ 실행결과



8.6 행렬 연산을 통한 기하학 변환-어파인 변환

◆ 기하학 변환 수식이 행렬의 곱으로 표현 가능

❖ 크기변경

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} \alpha & 0 \\ 0 & \beta \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$

평행이동

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} x \\ y \end{bmatrix} + \begin{bmatrix} t_x \\ t_y \end{bmatrix}$$

회전

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$

❖ 어파인 변환 수식

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} \alpha_{11} & \alpha_{12} & \alpha_{13} \\ \alpha_{21} & \alpha_{22} & \alpha_{23} \end{bmatrix} \cdot \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

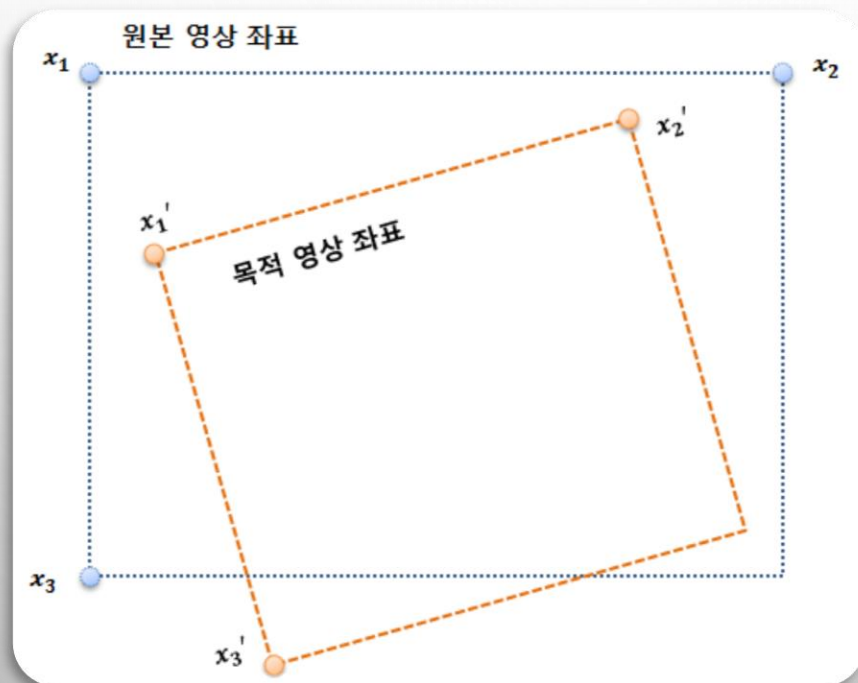
❖ 각 변환 행렬을 행렬 곱으로 구성가능 → 최종 행렬곱 후에 마지막 행 삭제

$$\text{어파인 변환행렬} = \begin{bmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} \alpha & 0 & 0 \\ 0 & \beta & 0 \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix}$$

8.6 행렬 연산을 통한 기하학 변환-어파인 변환

◆ 어파인 변환

- ❖ 기하학적 변환의 일종으로, 평행이동, 크기변경, 회전 등의 조합으로 만들수 있는 변환을 말함
- ❖ 입력 영상의 좌표 (x_1, x_2, x_3) 와 목적영상에서 상응하는 좌표 (x_1', x_2', x_3') 를 알면 → 어파인 행렬 구성 가능



8.6 행렬 연산을 통한 기하학 변환-어파인 변환

❖ OpenCV에서도 어파인 변환 수행 함수 제공

함수 및 인수 구조

`cv2.warpAffine (src, M, dsize [, dst [, flags [, borderMode [, borderValue ]]])` → `dst`

■ 설명: 입력 영상에 어파인 변환을 수행해서 반환한다.

인수 설명	■ src	입력 영상
	■ dst	반환 영상
	■ M	어파인 변환 행렬
	■ dsize	반환 영상의 크기
	■ flags	보간 방법
	■ borderMode	경계지정 방법

`cv2.getAffineTransform(src, dst)` → `retval`

■ 설명: 3개의 좌표쌍을 입력하면 어파인 변환 행렬을 반환한다.

인수 설명	■ src	입력 영상 좌표 3개 (행렬로 구성)
	■ dst	목적 영상 좌표 3개 (행렬로 구성)

`cv2.getRotationMatrix2D(center, angle, scale)` → `retval`

■ 설명: 회전 변환과 크기 변경을 수행할 수 있는 어파인 행렬을 반환한다.

인수 설명	■ center	회전의 중심점
	■ angle	회전각도, 양수 각도가 반시계 방향 회전 수행
	■ scale	변경할 크기

`cv2.invertAffineTransform(M [, iM])` → `iM`

■ 설명: 어파인 변환 행렬의 역 행렬을 반환한다.

인수 설명	■ M	어파인 변환 행렬
	■ iM	어파인 역변환 행렬

8.6 행렬 연산을 통한 기하학 변환-어파인 변환

예제 8.6.1 어파인 변환 - 07.affine_transform.py

```
01 import numpy as np, cv2
02 from Common.functions import contain          # 함수 импорт
03 from Common.interpolation import bilinear_value # 화소 선형 변환 함수 импорт
04
05 def affine_transform(img, mat):                # 어파인 변환 수행 함수
06     rows, cols = img.shape[:2]
07     inv_mat = cv2.invertAffineTransform(mat)   # 어파인 변환의 역행렬
08     ## 리스트 생성 방식
09     pts = [np.dot(inv_mat, (j, i, 1)) for i in range(rows) for j in range(cols)]
10     dst = [bilinear_value(img, p) if contain(p, size) else 0 for p in pts]
11     dst = np.reshape(dst, (rows, cols)).astype('uint8') # 1차원 → 2차원
12
13     ## 반복문 방식
14     # dst = np.zeros(img.shape, img.dtype)        # 목적 영상 생성
15     # for i in range(rows):                        # 목적 영상 순회-역방향 사상
16     #     for j in range(cols):
17     #         pt = np.dot(inv_mat, (j, i, 1))      # 행렬 내적 계산
18     #         if contain(pt, size): dst[i, j] = bilinear_value(img, pt) # 양선형 보간
19
20     return dst
```

8.6 행렬 연산을 통한 기하학 변환-어파인 변환

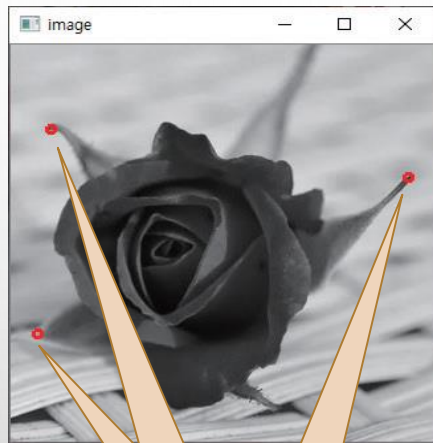
```
22 image = cv2.imread("images/affine.jpg", cv2.IMREAD_GRAYSCALE)
23 if image is None: raise Exception("영상파일 읽기 에러")
24
25 center = (200, 200)                                # 회전 변환 기준 좌표
26 angle, scale = 30, 1                                # 30도 회전, 크기 변경 안 함
27 size = image.shape[: -1]                             # 크기는 행태의 역순
28
29 pt1 = np.array([( 30, 70),(20, 240), (300, 110)], np.float32)
30 pt2 = np.array([(120, 20),(10, 180), (280, 260)], np.float32)
31 aff_mat = cv2.getAffineTransform(pt1, pt2)           # 3개 좌표쌍 어파인 행렬 생성
32 rot_mat = cv2.getRotationMatrix2D(center, angle, scale) # 어파인 행렬
33
34 dst1 = affine_transform(image, aff_mat)               # 어파인 변환 수행
35 dst2 = affine_transform(image, rot_mat)               # 회전 변환 수행
36 dst3 = cv2.warpAffine(image, aff_mat, size, cv2.INTER_LINEAR)
37 dst4 = cv2.warpAffine(image, rot_mat, size, cv2.INTER_LINEAR)
```

8.6 행렬 연산을 통한 기하학 변환-어파인 변환

```
39 image = cv2.cvtColor(image, cv2.COLOR_GRAY2BGR)
40 dst1 = cv2.cvtColor(dst1, cv2.COLOR_GRAY2BGR )
41 dst3 = cv2.cvtColor(dst3, cv2.COLOR_GRAY2BGR )
42
43 for i in range(len(pt1)):                # 3개 좌표 표시
44     cv2.circle(image, tuple(pt1[i].astype(int)), 3, (0, 0, 255), 2)
45     cv2.circle(dst1 , tuple(pt2[i].astype(int)), 3, (0, 0, 255), 2)
46     cv2.circle(dst3 , tuple(pt2[i].astype(int)), 3, (0, 0, 255), 2)
47
48 cv2.imshow("image", image)
49 cv2.imshow("dst1_affine", dst1);          cv2.imshow("dst2_affine_rotate", dst2)
50 cv2.imshow("dst3_OpenCV_affine", dst3);   cv2.imshow("dst4_OpenCV_affine_rotate", dst4)
51 cv2.waitKey(0)
```

8.6 행렬 연산을 통한 기하학 변환-어파인 변환

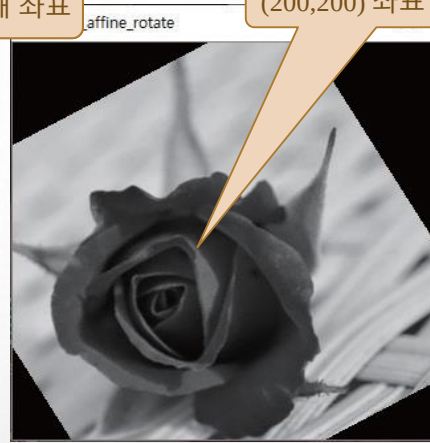
◆ 실행결과



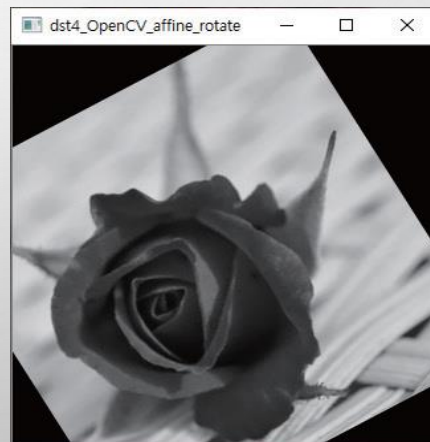
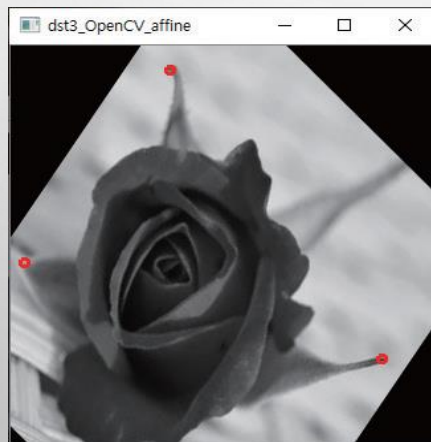
입력영상 3개 좌표



목적영상 3개 좌표



(200,200) 좌표 기준 회전



8.6 행렬 연산을 통한 기하학 변환-어파인 변환

심화예제 8.6.2 어파인 변환의 연결 - 07.affine_combination.py

```
01 import numpy as np, math, cv2
02 from Common.interpolation import affine_transform # 저자 구현 어파인변환 함수 임포트
03
04 def getAffineMat(center, degree, fx=1, fy=1, translate=(0,0)): # 변환 행렬 합성 함수
05     scale_mat = np.eye(3, dtype=np.float32) # 크기 변경 행렬
06     cen_trans = np.eye(3, dtype=np.float32) # 중점 평행 이동
07     org_trans = np.eye(3, dtype=np.float32) # 원점 평행 이동
08     trans_mat = np.eye(3, dtype=np.float32) # 좌표 평행 이동
09     rot_mat = np.eye(3, dtype=np.float32) # 회전 변환 행렬
10
11     radian = degree / 180 * np.pi # 회전 각도- 라디언 계산
12     rot_mat[0] = [ np.cos(radian), np.sin(radian), 0] # 회전행렬 0행
13     rot_mat[1] = [-np.sin(radian), np.cos(radian), 0] # 회전행렬 1행
14
15     cen_trans[:2, 2] = center # 중심 좌표 이동
16     org_trans[:2, 2] = -center[0], -center[1] # 원점으로 이동
17     trans_mat[:2, 2] = translate # 평행 이동 행렬의 원소 지정
18     scale_mat[0, 0], scale_mat[1, 1] = fx, fy # 크기 변경 행렬의 원소 지정
19
20     ret_mat = cen_trans.dot(rot_mat.dot(trans_mat.dot(scale_mat.dot(org_trans))))
21     # ret_mat = cen_trans.dot(rot_mat.dot(scale_mat.dot(trans_mat.dot(org_trans))))
22     return np.delete(ret_mat, 2, axis=0) # 마지막행 제거 ret_mat[0:2:]
```

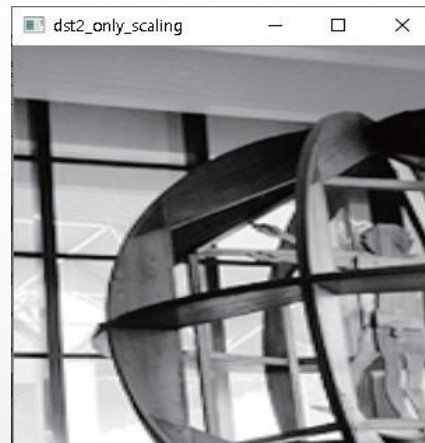
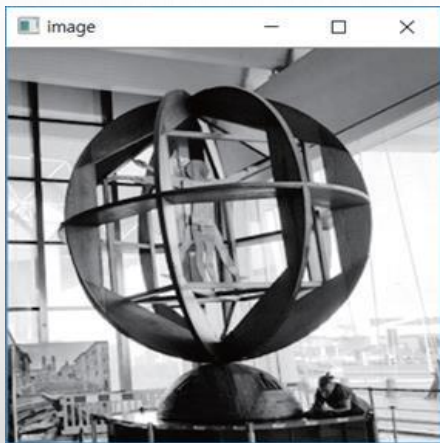

8.6 행렬 연산을 통한 기하학 변환-어파인 변환

```
24 image = cv2.imread("images/affine2.jpg", cv2.IMREAD_GRAYSCALE)
25 if image is None: raise Exception("영상파일 읽기 에러")
26
27 size = image.shape[::-1]
28 center = np.divmod(size, 2)[0]          # 회전 중심 좌표
29 angle, tr = 45, (200, 0)               # 각도와 평행이동 값 지정
30
31 aff_mat1 = getAffineMat(center, angle)  # 중심 좌표 기준 회전
32 aff_mat2 = getAffineMat((0,0), 0, 2.0, 1.5) # 크기 변경-확대
33 aff_mat3 = getAffineMat(center, angle, 0.7, 0.7) # 회전 및 축소
34 aff_mat4 = getAffineMat(center, angle, 0.7, 0.7, tr) # 복합 변환
35
36 dst1 = cv2.warpAffine(image, aff_mat1, size) # OpenCV 함수
37 dst2 = cv2.warpAffine(image, aff_mat2, size)
38 dst3 = affine_transform(image, aff_mat3)    # 사용자 정의 함수
39 dst4 = affine_transform(image, aff_mat4)
40
41 cv2.imshow("image", image)
42 cv2.imshow("dst1_only_rotate", dst1)
43 cv2.imshow("dst2_only_scaling", dst2)
44 cv2.imshow("dst3_rotate_scaling", dst3)
45 cv2.imshow("dst4_rotate_scaling_translate", dst4)
```

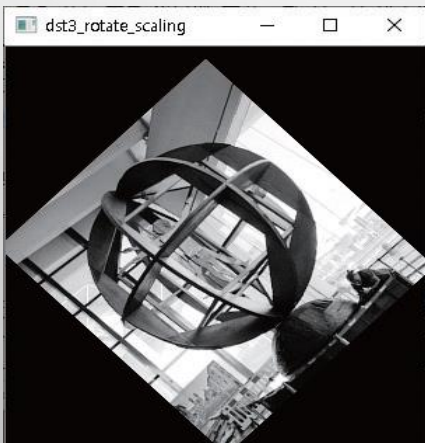
8.6 행렬 연산을 통한 기하학 변환-어파인 변환

◆ 실행결과

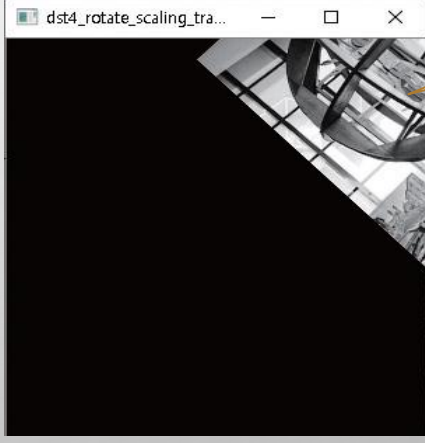
중심정 기준 회전



중심점에서 45도 회전
70%로 축소



중심점에서 45도 회전
70% 축소
x축으로 100 평행이동



심화예제

심화예제 8.6.3 마우스 드래그로 영상 회전하기 - 09.affine_event.py

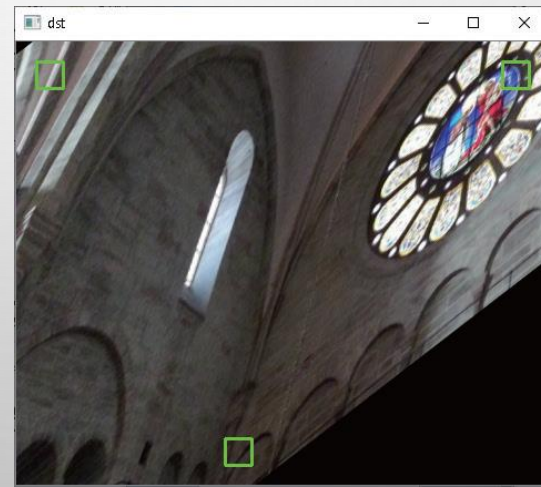
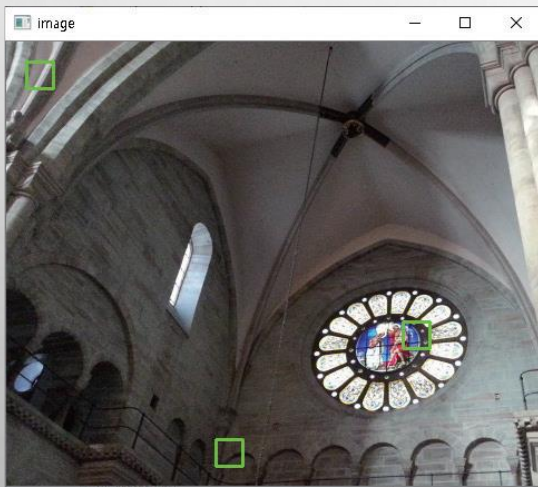
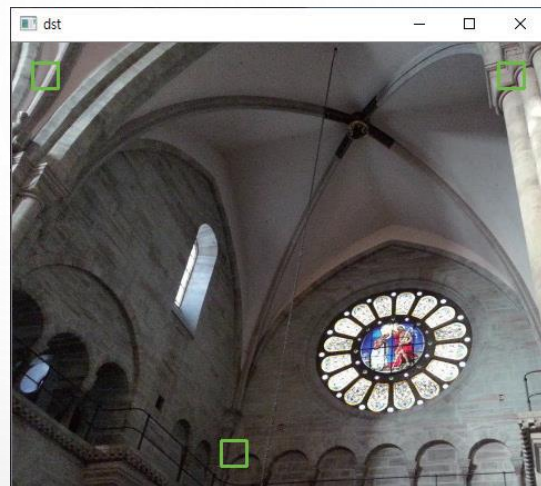
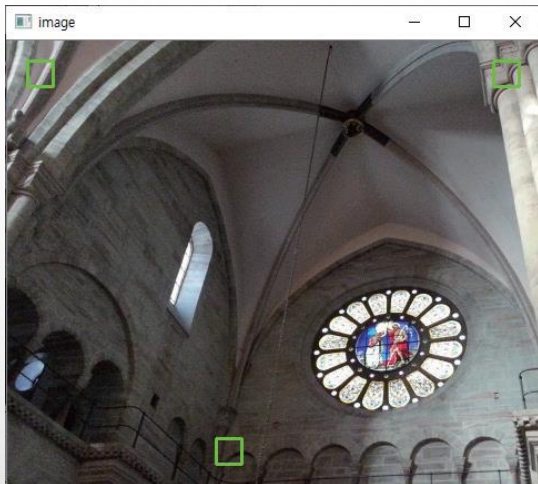
```
01  import numpy as np, cv2
02
03  def contain_pts(p, p1, p2):                                # p가 2개 좌표 범위 내 검사
04      return p1[0] <= p[0] < p2[0] and p1[1] <= p[1] < p2[1]
05
06  def draw_rect(title, img, pts):                            # 4개 사각형 표시 함수
07      rois = [(p - small, small * 2) for p in pts]          # 좌표 사각형 관심 영역들
08      for (x, y), (w, h) in np.int32(rois):
09          cv2.rectangle(img, (x, y, w, h), (0, 255, 0), 2)  # 사각형 그리기
10      cv2.imshow(title, img)
11
12  def affine(img):                                           # 어파인 변환 수행 함수
13      aff_mat = cv2.getAffineTransform(pts1, pts2)
14      dst = cv2.warpAffine(img, aff_mat, image.shape[1::-1], cv2.INTER_LINEAR)
15      draw_rect('image', np.copy(image), pts1)              # 입력 영상에 좌표 표시
16      draw_rect('dst', dst, pts2)                            # 목적 영상에 좌표 표시
```

```
18 def onMouse(event, x, y, flags, param):           # 마우스 이벤트 처리 함수
19     global check
20     if event == cv2.EVENT_LBUTTONDOWN:           # 좌버튼 다운
21         for i, p in enumerate(pts1):
22             p1, p2 = p - small, p + small         # small 2개 크기 좌표
23             if contain_pts((x, y), p1, p2): check = i     # 범위 내인지 확인
24
25     if event == cv2.EVENT_LBUTTONUP: check = -1     # 우버튼 업
26
27     if check >= 0:
28         pts1[check] = (x, y)
29         affine(np.copy(image))
```

```
31 image = cv2.imread("images/affine1.jpg", 1)       # 컬러 영상 읽기
32 if image is None: raise Exception("영상파일 읽기 에러")
33
34 small = np.array([12, 12])                         # 좌표 사각형 크기
35 check = -1                                           # 선택된 좌표 사각형
36 pts1 = np.float32([(30, 30), (450, 30), (200, 370)]) # 입력 영상 3개 좌표
37 pts2 = np.float32([(30, 30), (450, 30), (200, 370)]) # 목적 영상 3개 좌표
38
39 draw_rect('image', np.copy(image), pts1)           # 입력 영상에 좌표 사각형 그리기
40 draw_rect('dst', np.copy(image), pts2)             # 결과 영상에 좌표 사각형 그리기
41 cv2.setMouseCallback("image", onMouse, 0)
42 cv2.waitKey(0)
```

심화에제

◆ 실행결과



8.7 원근 투시(투영) 변환

◆ 아테네 학당

- ❖ 라파엘로 산치오 작품
- ❖ 바티칸 사도궁전의 방들 중에서 서명실에 그린 벽화
- ❖ 벽면에 그려진 그림에서 입체감 느끼는 이유 → 원근법



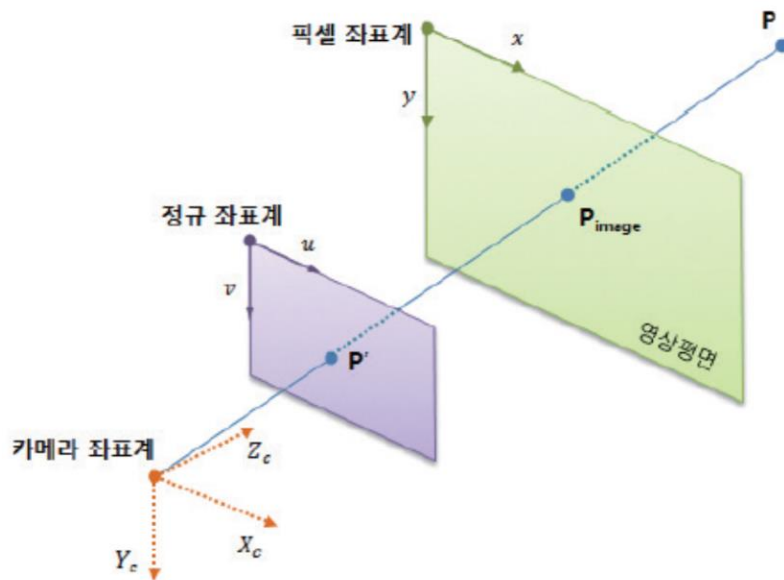
◆ 원근법

- ❖ 눈에 보이는 3차원의 세계를 2차원의 그림(평면)으로 옮길 때
- ❖ 관찰자가 보는 것 그대로 사물과의 거리를 반영하여 그리는 방법

8.7 원근 투시(투영) 변환

◆ 원근 투시 변환(perspective projection transformation)

- ❖ 이 원근법을 영상 좌표계에서 표현하는 것
- ❖ 3차원의 실세계 좌표를 투영 스크린상의 2차원 좌표로 표현할 수 있도록 변환해 주는 것



〈그림 8.7.2〉 3차원의 실세계 좌표를 2차원 좌표계에 표현 방법

8.7 원근 투시(투영) 변환

◆ 동차 좌표계(homogeneous coordinates)

- ❖ 모든 항의 차수가 동일하기 때문에 붙여진 이름

$$f_1(x_0, x_1) = x_0^3 + 3x_0^2x_1 - x_0x_1^2 + 2x_1^3$$

- ❖ n차원의 투영 공간을 n+1개의 좌표로 나타내는 좌표계

- 직교 좌표인 (x, y) 를 $(x, y, 1)$ 로 표현하는 것
- 일반화해서 0이 아닌 상수 ω 에 대해 (x, y) 를 $(\omega x, \omega y, \omega)$ 로 표현
- 상수 ω 가 무한히 많기 때문에 (x, y) 에 대한 동차 좌표 표현은 무한히 많이 존재

- ❖ 동차 좌표계에서 한 점 $(\omega x, \omega y, \omega)$ 을 직교 좌표로 나타내면

- 각 원소를 ω 로 나누어 $(x/\omega, y/\omega)$ 가 됨
- 예, 동차 좌표계에서 한 점 $(5, 7, 5) \rightarrow$ 직교 좌표에서 $(5/5, 7/5)$ 즉, $(1, 1.4)$

8.7 원근 투시(투영) 변환

◆ 원근 변환을 수행하는 행렬

$$w \cdot \begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} \alpha_{11} & \alpha_{12} & \alpha_{13} \\ \alpha_{21} & \alpha_{22} & \alpha_{23} \\ \alpha_{31} & \alpha_{32} & \alpha_{33} \end{bmatrix} \cdot \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

◆ OpenCV 함수

❖ cv2.getPerspectiveTransform(src, dst) 함수 → M

- 4개의 좌표쌍으로부터 원근변환 행렬 계산

❖ cv2.warpPerspective(src, M, dsize) 함수

- 원근 변환 행렬에 따라서 원근변환 수행

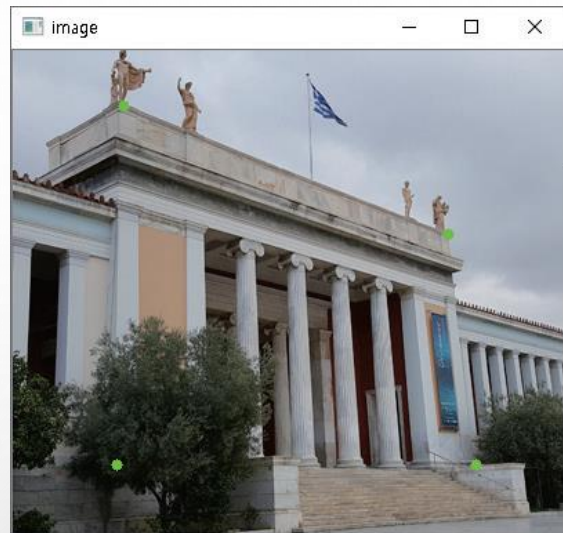
❖ cv2.transform(src, M) 함수

- 입력 영상의 4개 좌표와 원근 행렬을 인수로 입력하면 원근 변환된 좌표 반환

8.7 원근 투시(투영) 변환

심화예제 8.7.1 원근 왜곡 보정 - 10.perspective_transform.py

```
01 import numpy as np, cv2
02
03 image = cv2.imread("images/perspective.jpg", cv2.IMREAD_COLOR)
04 if image is None: raise Exception("영상파일 읽기 에러")
05
06 pts1 = np.float32([(80, 40), (315, 133), (75, 300), (335, 300)]) # 입력 영상 4개 좌표
07 pts2 = np.float32([(50, 60), (340, 60), (50, 320), (340, 320)]) # 목적 영상 4개 좌표
08
09 perspect_mat = cv2.getPerspectiveTransform(pts1, pts2) # 원근 변환 행렬
10 dst = cv2.warpPerspective(image, perspect_mat, image.shape[1::-1], cv2.INTER_CUBIC)
11 print("[perspect_mat] = \n%s\n" % perspect_mat )
```



8.7 원근 투시(투영) 변환

```
13 ## 변환 좌표 계산 - 행렬 내적 이용 방법
14 ones = np.ones((4,1), np.float64)
15 pts3 = np.append(pts1, ones, axis=1)           # 원본 좌표→동차 좌표 저장
16 pts4 = cv2.gemm(pts3, perspect_mat.T, 1, None, 1) # 좌표 변환값 계산
17
18 ## 변환 좌표 계산 - cv2.transform() 함수 이용방법
19 # pts3 = np.expand_dims(pts1, axis=0)           # 차원 증가
20 # pts4 = cv2.transform(pts3, m=perspect_mat)
21 # pts4 = np.squeeze(pts4, axis=0)               # 차원 감소
22 # pts3 = np.squeeze(pts3, axis=0)              # 차원 감소 - 결과 표시 위해
23
```

$$w \cdot \begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{13} \end{bmatrix} \cdot \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} \Rightarrow w \cdot \begin{bmatrix} x'_0 & y'_0 & 1 \\ x'_1 & y'_1 & 1 \\ x'_2 & y'_2 & 1 \\ x'_3 & y'_3 & 1 \end{bmatrix} = \begin{bmatrix} x_0 & y_0 & 1 \\ x_1 & y_1 & 1 \\ x_2 & y_2 & 1 \\ x_3 & y_3 & 1 \end{bmatrix} \cdot \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{13} \end{bmatrix}^T$$

8.7 원근 투시(투영) 변환

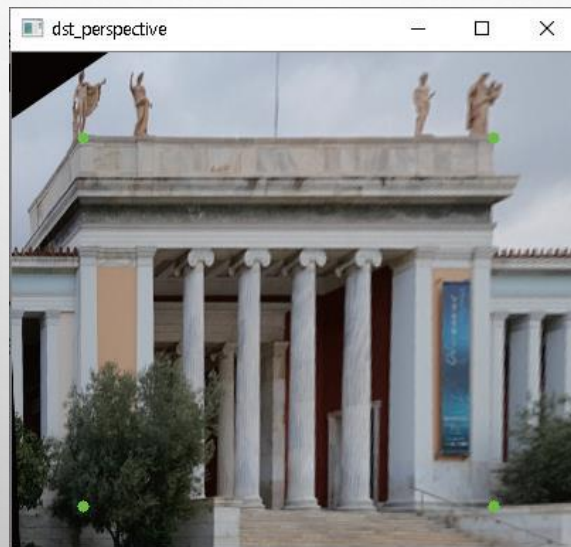
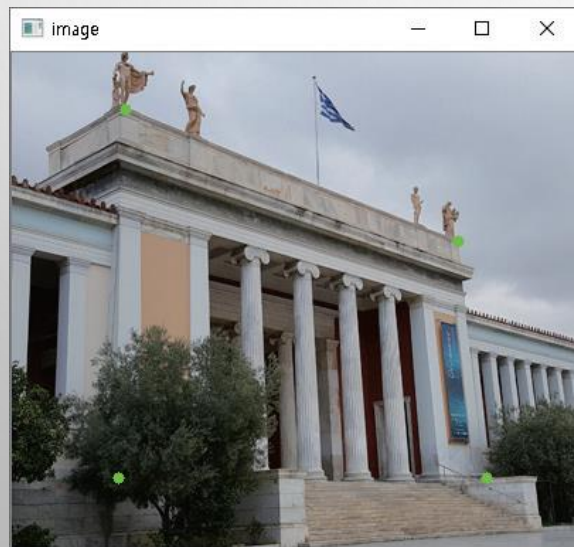
```
24 print(" 원본 영상 좌표 \t 목적 영상 좌표 \t\t 동차 좌표 \t\t 변환 결과 좌표")
25 for i in range(len(pts4)):
26     pts4[i] /= pts4[i][2]
27     print("%i : %-14s %-14s %-18s %-18s" % (i, pts1[i], pts2[i], pts3[i], pts4[i]))
28     cv2.circle(image, tuple(pts1[i].astype(int)), 3, (0, 255, 0), -1)
29     cv2.circle(dst, tuple(pts2[i].astype(int)), 3, (0, 255, 0), -1)
30
31 cv2.imshow("image", image)
32 cv2.imshow("dst_perspective", dst)
33 cv2.waitKey(0)
```


8.7 원근 투시(투영) 변환

◆ 실행결과

```
Run: 10.perspective_transform
C:\Python\python.exe D:/source/chap08/10.perspective_transform.py
[perspect_mat] =
[[ 6.25789284e-01  3.98298577e-02 -6.88839366e+00]
 [-5.02676539e-01  1.06358288e+00  5.13923399e+01]
 [-1.57086418e-03  5.25700042e-04  1.00000000e+00]]

원본 영상 좌표   목적 영상 좌표   동차 좌표   변환 결과 좌표
0 : [80. 40.]   [50. 60.]   [80. 40. 1.]   [50. 60. 1.]
1 : [315. 133.] [340. 60.]   [315. 133. 1.] [340. 60. 1.]
2 : [ 75. 300.]   [ 50. 320.]   [ 75. 300. 1.] [ 50. 320. 1.]
3 : [335. 300.]   [340. 320.]   [335. 300. 1.] [340. 320. 1.]
```



8.7 원근 투시(투영) 변환 - 심화예제

◆ 마우스 드래그로 선택된 영역에 원근 변환 제거하기

심화예제 8.7.2 마우스 이벤트로 원근 왜곡 보정 - 11.perspective_event.py

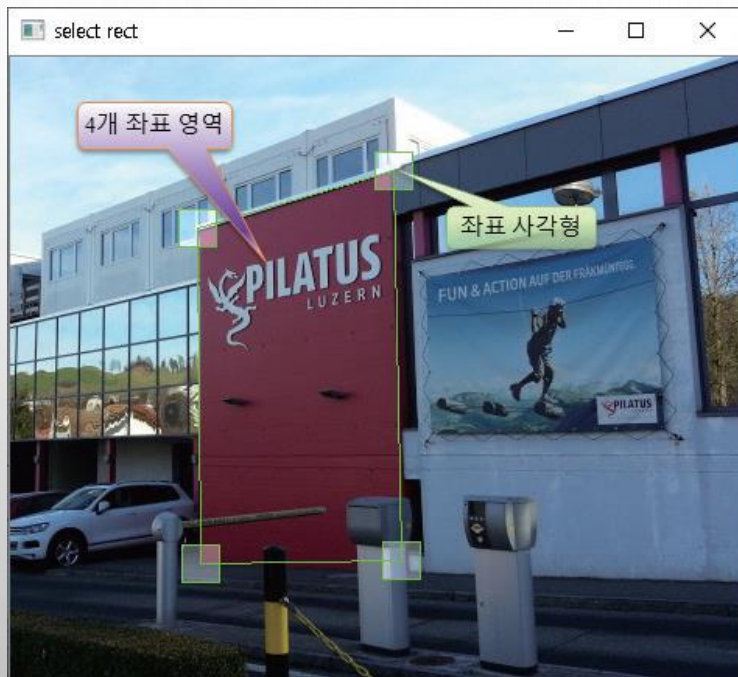
```
01 import numpy as np, cv2
02 from Common.functions import imread, contain_pts    # 좌표로 범위 확인 함수 импорт
03
04 def draw_rect(img):    # 좌표 사각형 그리기 함수
05     rois = [(p - small, small * 2) for p in pts1]    # 좌표 사각형 관심 영역
06     for (x, y), (w, h) in np.int32(rois):
07         roi = img[y:y+h, x:x+w]    # 좌표 사각형 범위 가져오기
08         val = np.full(roi.shape, 80, np.uint8)    # 컬러(3차원) 행렬 생성
09         cv2.add(roi, val, roi)    # 관심영역 밝기 증가
10         cv2.rectangle(img, (x, y, w, h), (0, 255, 0), 1)
11     cv2.polylines(img, [pts1.astype(int)], True, (0, 255, 0), 1) # 4개 좌표 잇기
12     cv2.imshow("select rect", img)
13
14 def warp(img):    # 원근 변환 수행 함수
15     perspect_mat = cv2.getPerspectiveTransform(pts1, pts2)
16     dst = cv2.warpPerspective(img, perspect_mat, (400, 350), cv2.INTER_CUBIC) # 원근 변환
17     cv2.imshow("perspective transform", dst)
```

8.7 원근 투시(투영) 변환

```
19 def onMouse(event, x, y, flags, param): # 마우스 이벤트 처리 함수
20     global check
21     if event == cv2.EVENT_LBUTTONDOWN:
22         for i, p in enumerate(pts1):
23             p1, p2 = p - small, p + small # p 좌표의 위상단, 좌하단 좌표생성
24             if contain_pts((x,y), p1, p2): check = i # 클릭 좌표로 좌표 사각형 선택
25
26     if event == cv2.EVENT_LBUTTONUP: check = -1 # 마우스 업시 좌표번호 초기화
27
28     if check >= 0 : # 좌표 사각형 선택 시
29         pts1[check] = (x, y)
30         draw_rect(np.copy(image))
31         warp(np.copy(image))
32
33 image = cv2.imread('images/perspective2.jpg', cv2.IMREAD_COLOR)
34 if image is None: raise Exception("영상파일 읽기 에러")
35
36 small = np.array([12, 12]) # 좌표 사각형 크기
37 check = -1 # 선택 좌표 사각형 번호 초기화
38 pts1 = np.float32([(100, 100), (300, 100), (300, 300), (100, 300)]) # 4개 좌표 초기화
39 pts2 = np.float32([(0, 0), (400, 0), (400, 350), (0, 350)]) # 목적 영상 4개 좌표
40
41 draw_rect(np.copy(image))
42 cv2.setMouseCallback("select rect", onMouse, 0)
43 cv2.waitKey(0)
```

8.7 원근 투시(투영) 변환

◆ 실행결과



단원 요약

- ◆ 1. 사상(mapping)은 화소들의 배치를 변경할 때, 입력 영상의 좌표가 새롭게 배치될 해당 목적 영상의 좌표를 찾아서 화소값을 옮기는 과정을 말한다. 순방향 사상(forward mapping)과 역방향 사상(reverse mapping)의 두 가지 방식이 있다.
- ◆ 2. 사상의 과정에서 홀(hole)과 오버랩(overlap)이 발생할 수 있다. 홀은 입력 영상의 좌표들로 목적 영상의 좌표를 만드는 과정에서 사상되지 않은 화소이다. 오버랩은 원본 영상의 여러 화소들이 목적 영상의 한 화소로 사상되는 것을 말한다.
- ◆ 3. 목적 영상에서 홀의 화소들을 채우고, 오버랩이 되지 않게 화소들을 배치하여 목적 영상을 만드는 기법을 보간법(interpolation)이라 하며, 그 종류에는 최근접 이웃 보간법, 양선형 보간법, 3차 회선 보간법 등 다양한 방법들이 있다.
- ◆ 4. 최근접 이웃 보간법은 목적 영상을 만드는 과정에서 홀이 되어 할당받지 못한 화소들의 값을 찾을 때, 목적 영상의 화소에 가장 가깝게 이웃한 입력 영상의 화소값을 가져오는 방법이다. 양선형 보간법은 선형 보간을 두 번에 걸쳐서 수행하기에 붙여진 이름이다.

단원 요약

- ◆ 5. OpenCV에서는 `cv2.resize()`, `cv2.remapping()`, `cv2.warpAffine()`, `cv2.warpPerspective()` 등과 같이 영상을 변환하는 함수에서 보간을 위한 `flag` 옵션을 제공한다. 대표적으로 'cv2.INTER_NEAREST'는 최근접 이웃 보간이며, 'cv2.INTER_LINEAR'는 양선형 보간이며, 'cv2.INTER_CUBIC'는 바이큐빅 보간이다.
- ◆ 6. 2행, 3열의 어파인 변환 행렬을 이용해서 회전, 크기변경, 평행이동 등을 복합적으로 수행할 수 있다. OpenCV에서는 `cv2.getAffineTransform()` 와 `getRotationMatrix2D()` 함수로 어파인 변환 행렬을 만들며, `cv2.warpAffine()` 함수로 어파인 변환을 수행한다.
- ◆ 7. 원근법은 눈에 보이는 3차원의 세계를 2차원의 평면으로 옮길 때 관찰자가 보는 것 그대로 사물과의 거리를 반영하여 그리는 방법을 말한다. 그리고 이 원근법을 영상 좌표계에서 표현하는 것이 원근 투시 변환이다. 원근 변환에서는 주로 동차 좌표계를 사용하는 것이 편리하다.
- ◆ 8. OpenCV에서는 `cv2.getPerspectiveTransform()` 함수로 원근 변환 행렬을 계산하며, `cv2.warpPerspective()` 함수는 원근 변환 행렬에 따라서 원근 변환을 수행한다. 또한, `cv2.transform()` 함수는 입력 영상의 4개 좌표와 원근 행렬을 인수로 입력하면 원근 변환된 좌표를 반환해 준다.